

LAPORAN TUGAS
STURKTUR DATA DAN ALGORITMA
PENJELASAN TAMBAHAN TUGAS 4 ALGORITMA SORTING



Dosen Pengampu:

Prof. Dr. Taufik Fuadi Abidin, M.Tech
Dr. Ir. Irvanizam, S.Si., M.Sc., IPM., ASEAN Eng.

Disusun oleh:

Muhammad Aqil Mubarak (250810701100003)

JURUSAN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
BANDA ACEH
2026

PROBLEM TUGAS 4

Problem 1

Membuat program untuk mengurutkan bilangan bulat menggunakan metode bubble sort, selection sort, insertion sort, merge sort, dan quick sort. Anda dimintakan untuk membuat sebuah program yang dapat mengurutkan data berupa bilangan desimal menggunakan metode bubble sort, selection sort, insertion sort, merge sort, dan quick sort. Program menerima input berupa n sebuah bilangan dimana $1 < n \leq 5000000$ (5 juta bilangan) dan kemudian mengacak n buah bilangan bulat. Program yang dibuat harus menyediakan MENU dan harus melakukan hal-hal sebagai berikut (sebagai contoh):

Masukkan Jumlah Bilangan (n): 100000

Random data dalam array selesai

Pilihan:

- 1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil*
- 2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil*
- 3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil*
- 4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil*
- 5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil*
- 6) Random ulang data dalam array*
- 7) Selesai*

Pilihan anda: _

Gunakan fungsi `rand()` untuk mengacak bilangan bulat tersebut. Info tentang `rand()` dapat diperoleh di Internet, salah satunya pada URL berikut.

<http://www.cplusplus.com/reference/cstdlib/rand/>

Selanjutnya, program akan menampilkan menu dengan beberapa pilihan. Bila pilihan (1) dipilih, maka program akan mengurutkan data menggunakan metode insertion sort dan menampilkan data yang telah terurut dari kecil ke besar, menampilkan jumlah perbandingan dan pertukaran data, serta memperlihatkan jumlah waktu yang dibutuhkan untuk mengurutkan data tersebut dalam satuan detik atau milidetik.

Bila pilihan (2) yang dipilih, maka program akan mengurutkan data menggunakan metode bubble sort dan menampilkan data yang telah terurut dari kecil ke besar, menampilkan jumlah perbandingan dan pertukaran data, serta memperlihatkan jumlah waktu yang dibutuhkan untuk mengurutkan data tersebut dalam satuan detik atau milidetik.

Bila pilihan (3) yang dipilih, maka program akan mengurutkan data menggunakan metode selection sort dan menampilkan data yang telah terurut dari kecil ke besar, menampilkan jumlah perbandingan dan pertukaran data, serta memperlihatkan jumlah waktu yang dibutuhkan untuk mengurutkan data tersebut dalam satuan detik atau milidetik.

Bila pilihan (4) yang dipilih, maka program akan mengurutkan data menggunakan metode merge sort dan menampilkan data yang telah terurut dari kecil ke besar, menampilkan jumlah perbandingan dan pertukaran data, serta memperlihatkan jumlah waktu yang dibutuhkan untuk mengurutkan data tersebut dalam satuan detik atau milidetik.

Bila pilihan (5) yang dipilih, maka program akan mengurutkan data menggunakan metode quick sort dan menampilkan data yang telah terurut dari kecil ke besar, menampilkan jumlah perbandingan dan pertukaran data, serta memperlihatkan jumlah waktu yang dibutuhkan untuk mengurutkan data tersebut dalam satuan detik atau milidetik.

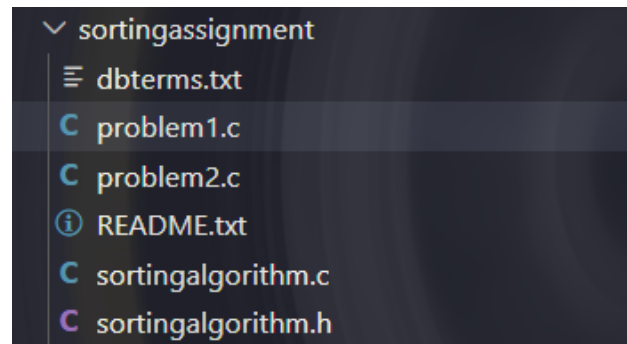
Bila pilihan (6) yang dipilih, maka program akan meminta input nilai n kembali dan mengacak kembali n bilangan bulat ke dalam array. Program akan berhenti bila Pilihan (7) dipilih.

Problem 2

Membuat program untuk mengurutkan kata-kata menggunakan metode bubble sort, selection sort, insertion sort, merge sort, dan quick sort. Input data dibaca dari file teks. Anda dimintakan untuk membuat sebuah program yang dapat mengurutkan data berupa kata kata dari file teks menggunakan metode bubble sort, selection sort, insertion sort, merge sort, dan quick sort. Program menerima input file teks dari URL berikut sebagai contoh <https://informatika.usk.ac.id/tfa/ds/dbterms.txt>. Kata-kata dalam file teks tersebut berjumlah 25988 (ada duplikasi) dan disusun secara tidak terurut. Program harus menyediakan MENU seperti pada soal 1, namun tanpa perlu ada fitur pengacakan bilangan.

PENJELASAN TAMBAHAN

Dalam Pengerjaan solusi problem ini, saya menggunakan pendekatan modularitas, dimana program dipecah menjadi beberapa file terpisah yaitu file header dan file implementasi. Cara ini memisahkan logika dasar struktur data dengan logika utama penyelesaian pada problem, sehingga kode menjadi rapi dan mudah di kelola.



Gambar 1. Direktori File

Program dapat di compile dengan command:

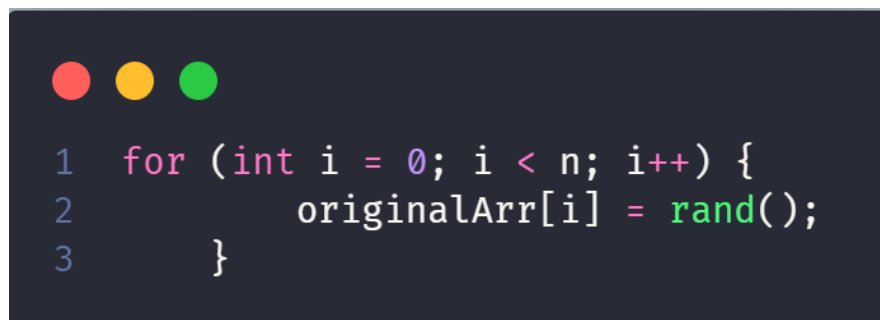
```
'gcc -Wall -pedantic -g -o problem1 problem1.c sortingalgorithm.c '
```

dan

```
'gcc -Wall -pedantic -g -o problem2 problem2.c sortingalgorithm.c '
```

Problem 1

Anda dimintakan untuk membuat sebuah program yang dapat mengurutkan data berupa bilangan desimal menggunakan metode bubble sort, selection sort, insertion sort, merge sort, dan quick sort. Program menerima input berupa n sebuah bilangan dimana $1 < n \leq 5000000$ (5 juta bilangan) dan kemudian mengacak n buah bilangan bulat.



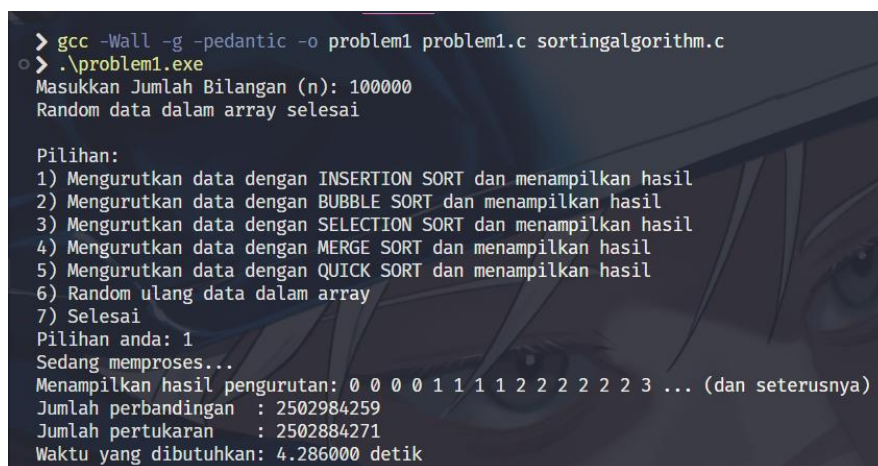
```
1  for (int i = 0; i < n; i++) {  
2      originalArr[i] = rand();  
3  }
```

Gambar 2. Implementasi fungsi random pada problem 1

Untuk menangani kapasitas komputasi yang sangat besar ini, penyelesaian utama difokuskan pada manajemen memori menggunakan alokasi memori dinamis (*dynamic memory allocation*) menggunakan fungsi `malloc()`. Dimana penggunaan array yang statis sangat dihindari untuk mencegah terjadinya *Stack Overflow* yang dapat membuat program tertutup secara paksa (*crash*). Pada program ini ditugaskan menginisialisasi angka secara acak menggunakan fungsi `rand()` yang dipadukan dengan `seed` `srand(time(NULL))` agar set data yang dihasilkan selalu bervariasi setiap kali program dijalankan. Program juga dilengkapi dengan validasi *input* untuk membersihkan *buffer* memori apabila pengguna secara tidak sengaja memasukkan karakter huruf, sehingga program tetap berjalan stabil tanpa mengalami *infinite loop*.

Untuk memastikan perbandingan efisiensi kelima algoritma berjalan secara adil, program memisahkan penyimpanan data ke dalam dua tempat, yaitu variabel array sumber (`original_arr`) dan array sementara (`temp_arr`). Setiap kali pengguna memilih metode pengurutan dari menu, program akan menyalin data dari array sumber ke array sementara. Hal ini menjamin bahwa setiap algoritma (Insertion, Bubble, Selection, Merge, dan Quick Sort) selalu mengurutkan kumpulan angka acak yang sama persis. Untuk pelaporan kinerjanya, program memanfaatkan sebuah tipe bentukan *struct* yang dilempar menggunakan *pointer* ke dalam fungsi algoritma untuk menghitung jumlah perbandingan (*comparison*) dan pertukaran (*swap*). Selain itu, fungsi `clock()` dari pustaka `<time.h>` digunakan sebagai *stopwatch* untuk mencatat total waktu eksekusi (dalam satuan detik) secara presisi.

Output Problem: Dengan jumlah bilangan (n) = 100000



```
> gcc -Wall -g -pedantic -o problem1 problem1.c sortingalgorithm.c
> ./problem1.exe
Masukkan Jumlah Bilangan (n): 100000
Random data dalam array selesai

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Random ulang data dalam array
7) Selesai
Pilihan anda: 1
Sedang memproses...
Menampilkan hasil pengurutan: 0 0 0 0 1 1 1 1 2 2 2 2 2 3 ... (dan seterusnya)
Jumlah perbandingan : 2502984259
Jumlah pertukaran : 2502884271
Waktu yang dibutuhkan: 4.286000 detik
```

Gambar 3. Output Problem 1 Pilihan 1 (Insertion Sort)

```

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Random ulang data dalam array
7) Selesai
Pilihan anda: 2
Sedang memproses...
Menampilkan hasil pengurutan: 0 0 0 0 1 1 1 1 2 2 2 2 2 3 ... (dan seterusnya)
Jumlah perbandingan : 5000049999
Jumlah pertukaran : 2502884271
Waktu yang dibutuhkan: 21.032000 detik

```

Gambar 4. Output Problem 1 Pilihan 2 (Bubble Sort)

```

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Random ulang data dalam array
7) Selesai
Pilihan anda: 3
Sedang memproses...
Menampilkan hasil pengurutan: 0 0 0 0 1 1 1 1 2 2 2 2 2 3 ... (dan seterusnya)
Jumlah perbandingan : 4999950000
Jumlah pertukaran : 99988
Waktu yang dibutuhkan: 6.020000 detik

```

Gambar 5. Output Problem 1 Pilihan 3 (Selection Sort)

```

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Random ulang data dalam array
7) Selesai
Pilihan anda: 4
Sedang memproses...
Menampilkan hasil pengurutan: 0 0 0 0 1 1 1 1 2 2 2 2 2 3 ... (dan seterusnya)
Jumlah perbandingan : 1536400
Jumlah pertukaran : 1668928
Waktu yang dibutuhkan: 0.022000 detik

```

Gambar 6. Output Problem 1 Pilihan 4 (Merge Sort)

```

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Random ulang data dalam array
7) Selesai
Pilihan anda: 5
Sedang memproses...
Menampilkan hasil pengurutan: 0 0 0 0 1 1 1 1 2 2 2 2 2 3 ... (dan seterusnya)
Jumlah perbandingan : 2486762
Jumlah pertukaran : 424581
Waktu yang dibutuhkan: 0.008000 detik

```

Gambar 7. Output Problem 1 Pilihan 5 (Quick Sort)


```

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Random ulang data dalam array
7) Selesai
Pilihan anda: 6
Masukkan Jumlah Bilangan (n): 100000
Random ulang data dalam array selesai

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Random ulang data dalam array
7) Selesai
Pilihan anda: 7
Program selesai.

```

Gambar 8. Output Problem 1 Pilihan 6 & 7

Memory Check Problem:

```

eruma@kali: ~/Downloads/SDA-main/sortingassignment
Session Actions Edit View Help
Masukkan Jumlah Bilangan (n): 100000
Random data dalam array selesai

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Random ulang data dalam array
7) Selesai
Pilihan anda: 7
Program selesai.

(eruma@kali)-[~/Downloads/SDA-main/sortingassignment]
$ valgrind ./problem1
==5530== Memcheck, a memory error detector
==5530== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==5530== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==5530== Command: ./problem1
==5530==
Masukkan Jumlah Bilangan (n): 100000
Random data dalam array selesai

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Random ulang data dalam array
7) Selesai
Pilihan anda: 7
Program selesai.
==5530==
==5530== HEAP SUMMARY:
==5530==   in use at exit: 0 bytes in 0 blocks
==5530==   total heap usage: 4 allocs, 4 frees, 802,048 bytes allocated
==5530==
==5530== All heap blocks were freed -- no leaks are possible
==5530==
==5530== For lists of detected and suppressed errors, rerun with: -s
==5530== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

(eruma@kali)-[~/Downloads/SDA-main/sortingassignment]
$

```

Gambar 9. Memory check menggunakan valgrind

Problem 2

Membuat program untuk mengurutkan kata-kata menggunakan metode bubble sort, selection sort, insertion sort, merge sort, dan quick sort. Input data dibaca dari file teks. Anda dimintakan untuk membuat sebuah program yang dapat mengurutkan data berupa kata-kata dari file teks menggunakan metode bubble sort, selection sort, insertion sort, merge sort, dan quick sort. Program menerima input file teks dari URL berikut sebagai contoh <https://informatika.usk.ac.id/tfa/ds/dbterms.txt>. Kata-kata dalam file teks tersebut berjumlah 25988 (ada duplikasi) dan disusun secara tidak teratur.



```
1 void bubbleSortStr (char arr[][100], int n, Sortstats *stats) {
2     char temp[100];
3
4     for (int i = 0; i < n - 1; i++) {
5         for (int j = 0; j < n - 1; j++) {
6             stats->comparison++;
7             if (strcmp(arr[j], arr[j+1]) > 0) {
8                 strcpy(temp, arr[j]);
9                 strcpy(arr[j], arr[j + 1]);
10                strcpy(arr[j + 1], temp);
11                stats->swap++;
12            }
13        }
14    }
15 }
```

Gambar 10. Implementasi bubble sort pada data string

Dalam penyelesaian terbagi beberapa tahap, pada tahap pertama yang dilakukan program adalah memindai isi file tersebut hingga mencapai akhir data EOF (*End of File*) menggunakan fungsi `fscanf()` untuk menghitung total baris kata secara otomatis. Dengan pendekatan ini, pengguna tidak perlu lagi memasukkan jumlah data (n) secara manual ke dalam terminal. Apabila file tidak ditemukan di dalam direktori, program telah dilengkapi dengan penanganan *error* untuk memberikan peringatan yang jelas kepada user.

Setelah dimensi data diketahui, memori dinamis kembali dialokasikan, tetapi disusun dalam bentuk *Array 2 Dimensi* (Array of Strings) untuk menampung puluhan ribu kata tersebut di dalam *Heap Memory*. Pada implementasi kelima algoritma pengurutannya, penyesuaian khusus dilakukan pada operator logika. Operator perbandingan matematika standar diganti dengan fungsi `strcmp()` untuk mengevaluasi urutan leksikografis (alfabet) dari dua buah kata. Selain itu, proses pertukaran data (*swap*) tidak lagi menggunakan operator penugasan (`=`), melainkan menggunakan fungsi `strcpy()` untuk memindahkan isi teks antar variabel sementara. Proses pencatatan statistik (waktu, perbandingan, dan pertukaran) tetap diintegrasikan di dalam

loop menu utama, memungkinkan perbandingan kinerja komputasi yang komprehensif antara pengurutan tipe data integer dan tipe data string.

Output Problem: Dengan data dari file dbterms.txt

```
●> gcc -Wall -g -pedantic -o problem2 problem2.c sortingalgorithm.c
○> .\problem2.exe
Berhasil membaca 25988 kata dari file dbterms.txt.
Data siap diproses!

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Selesai
Pilihan anda: 1
Sedang memproses 25988 kata...
Menampilkan hasil pengurutan:
aa aam aan aang aanharyono aasld aat aatu ab aba abad abadi abah abaikan abang ... (dan seterusnya)
Jumlah perbandingan : 167901951
Jumlah pertukaran : 167875977
Waktu yang dibutuhkan: 2.127000 detik
```

Gambar 11. Output Problem 2 Pilihan 1 (Insertion Sort)

```
Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Selesai
Pilihan anda: 2
Sedang memproses 25988 kata...
Menampilkan hasil pengurutan:
aa aam aan aang aanharyono aasld aat aatu ab aba abad abadi abah abaikan abang ... (dan seterusnya)
Jumlah perbandingan : 675324169
Jumlah pertukaran : 167875977
Waktu yang dibutuhkan: 10.563000 detik
```

Gambar 12. Output Problem 2 Pilihan 2 (Bubble Sort)

```
Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Selesai
Pilihan anda: 3
Sedang memproses 25988 kata...
Menampilkan hasil pengurutan:
aa aam aan aang aanharyono aasld aat aatu ab aba abad abadi abah abaikan abang ... (dan seterusnya)
Jumlah perbandingan : 337701065
Jumlah pertukaran : 25984
Waktu yang dibutuhkan: 0.867000 detik
```

Gambar 13. Output Problem 2 Pilihan 3 (Selection Sort)

```
Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Selesai
Pilihan anda: 4
Sedang memproses 25988 kata...
Menampilkan hasil pengurutan:
aa aam aan aang aanharyono aasld aat aatu ab aba abad abadi abah abaikan abang ... (dan seterusnya)
Jumlah perbandingan : 348590
Jumlah pertukaran : 383040
Waktu yang dibutuhkan: 0.033000 detik
```

Gambar 14. Output Problem 2 Pilihan 4 (Merge Sort)

```

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Selesai
Pilihan anda: 5
Sedang memproses 25988 kata...
Menampilkan hasil pengurutan:
aa aam aan aang aanharyono aasld aat aatu ab aba abad abadi abah abaikan abang ... (dan seterusnya)
Jumlah perbandingan : 607183
Jumlah pertukaran : 88756
Waktu yang dibutuhkan: 0.008000 detik

```

Gambar 15. Output Problem 2 Pilihan 5 (Quick Sort)

```

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Selesai
Pilihan anda: 6
Program selesai.

```

Gambar 16. Output Problem 2 Pilihan 6

Memory Check Problem:

```

eruma@kali: ~/Downloads/SDA-main/sortingassignment
Session Actions Edit View Help
==5530== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

(eruma@kali)~[~/Downloads/SDA-main/sortingassignment]
$ gcc -Wall -g -pedantic -o problem2 problem2.c sortingalgorithm.c

(eruma@kali)~[~/Downloads/SDA-main/sortingassignment]
$ valgrind ./problem2
==6912== Memcheck, a memory error detector
==6912== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==6912== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==6912== Command: ./problem2
==6912==
Berhasil membaca 25988 kata dari file dbterms.txt.
Data siap diproses!

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Selesai
Pilihan anda: 7
Pilihan tidak valid. Silakan masukkan angka 1 hingga 6.

Pilihan:
1) Mengurutkan data dengan INSERTION SORT dan menampilkan hasil
2) Mengurutkan data dengan BUBBLE SORT dan menampilkan hasil
3) Mengurutkan data dengan SELECTION SORT dan menampilkan hasil
4) Mengurutkan data dengan MERGE SORT dan menampilkan hasil
5) Mengurutkan data dengan QUICK SORT dan menampilkan hasil
6) Selesai
Pilihan anda: 6
Program selesai.
==6912==
==6912== HEAP SUMMARY:
==6912==   in use at exit: 0 bytes in 0 blocks
==6912==   total heap usage: 8 allocs, 8 frees, 5,208,784 bytes allocated
==6912==
==6912== All heap blocks were freed -- no leaks are possible
==6912==
==6912== For lists of detected and suppressed errors, rerun with: -s
==6912== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

(eruma@kali)~[~/Downloads/SDA-main/sortingassignment]
$

```

Gambar 17. Memory check menggunakan valgrind

KESIMPULAN

Berdasarkan hasil implementasi dan pengujian pada Tugas ini, dapat disimpulkan bahwa terdapat perbedaan efisiensi yang sangat signifikan antara algoritma pengurutan sederhana (Bubble, Selection, dan Insertion Sort) ber-kompleksitas $O(n^2)$ dibandingkan dengan algoritma lanjutan (Merge dan Quick Sort) ber-kompleksitas $O(n \log n)$, di mana Merge dan Quick Sort terbukti jauh lebih efektif diperlukan dalam menangani dataset yang besar. Total waktu eksekusi komputasi pada setiap metode terbukti berbanding lurus dengan jumlah perbandingan dan pertukaran data yang terjadi selama proses tersebut. Eksperimen ini juga menegaskan bahwa manajemen memori dinamis menggunakan alokasi *Heap* `malloc()` wajib diterapkan untuk mencegah *Stack Overflow* saat memproses jutaan data, meskipun algoritma sekelas Merge Sort akan menuntut alokasi memori tambahan yang cukup besar. Pada akhirnya, logika dasar algoritma pengurutan ini terbukti bersifat universal dan fleksibel, di mana pengurutan alfabetis untuk puluhan ribu data teks dapat dicapai secara efisien hanya dengan mengadaptasi operator evaluasi matematika menjadi fungsi `strcmp()` dan pertukaran memori menggunakan `strcpy()`.